

Problem 3: Movie Ticket Booking and Seat Management System

Overview of the System :-

The Movie Ticket Booking and Seat Management System is a Python-based application developed using Object-Oriented Programming (OOP) concepts. The system is designed to manage movie ticket bookings, seat availability, payment processing, customer offers, and booking records in an organized and secure way.

The system supports different customer categories such as Normal Customers, Student Customers, and Premium Customers. Each customer type receives different discounts and benefits according to predefined business rules. Customers can book movie tickets only when seats are available and payment is completed successfully.

The project also supports different payment behaviors while maintaining a common payment process. Seats are reduced only after successful booking confirmation and successful payment. If payment fails or seats are unavailable, the booking is marked as failed and the seat count remains unchanged.

All successful and failed booking records are saved into a file because failed bookings are also useful for business analysis. The system can generate reports such as:

- Total tickets booked
- Total booking revenue
- Movie-wise successful bookings
- Movie-wise failed bookings
- Remaining seats report

This project demonstrates practical implementation of important OOP concepts such as Classes and Objects, Inheritance, Polymorphism, Encapsulation, Abstraction, Exception Handling, and File Handling in a real-world cinema booking scenario.

Explanation of the Assignment :-

This project simulates a cinema ticket booking platform where customers can book movie tickets using their wallet balance. Different customer types receive different discounts based on their category. The system checks seat availability, validates ticket quantity, processes payment, updates available seats, and stores booking details.

Inheritance and method overriding are used to implement customer-based discount offers. Encapsulation is used to protect sensitive data such as wallet balance, ticket price, available seats, and booking status. Abstraction is used to simplify the booking process while hiding internal validation and payment steps from the user.

The system also handles invalid situations such as:

- Invalid seat quantity
- Seats not available
- Low wallet balance
- Wrong input type for seats

All booking details are saved into a file for future reporting and analysis. Overall, the project provides a practical and realistic implementation of a Movie Ticket Booking and Seat Management System using Python and OOP principles.

A6 -Practice Problems 3

May 26, 2026

1 Problem 3: Movie Ticket Booking and Seat Management System

2 CREATING CUSTOMER CLASS

```
[1]: class Customer:

    def __init__(self, name, wallet_balance):
        self.name = name
        self.__wallet_balance = wallet_balance

    def get_wallet_balance(self):
        return self.__wallet_balance

    def pay_from_wallet(self, amount):
        if amount <= self.__wallet_balance:
            self.__wallet_balance -= amount
            return True
        else:
            return False

    def get_discount_percentage(self):
        return 0
```

3 creating Special Customer Types

```
[2]: class StudentCustomer(Customer):
    def get_discount_percentage(self):
        return 10

class PremiumCustomer(Customer):
    def get_discount_percentage(self):
        return 20
```

4 Creating Movie Class

```
[3]: class Movie:
    def __init__(self, movie_name, ticket_price, seats):
        self.movie_name = movie_name
        self.__ticket_price = ticket_price
        self.__seats = seats

    def get_ticket_price(self):
        return self.__ticket_price

    def get_seats(self):
        return self.__seats

    def reduce_seats(self, quantity):
        if quantity <= self.__seats:
            self.__seats -= quantity
            return True
        else:
            return False

    def display_movie_details(self):
        print("\nMovie Details")
        print("Movie Name:", self.movie_name)
        print("Ticket Price:", self.__ticket_price)
        print("Available Seats:", self.__seats)
```

5 Creating Booking Class

```
[4]: class Booking:

    def __init__(self, customer, movie, quantity):
        self.customer = customer
        self.movie = movie
        self.quantity = quantity
        self.total_amount = 0
        self.discount_amount = 0
        self.final_amount = 0
        self.booking_status = "Pending"
        self.failure_reason = "None"

    def place_booking(self):
        try:
            quantity = int(self.quantity)
            if quantity <= 0:
                self.booking_status = "Failed"
```

```

        self.failure_reason = "Invalid Seat Quantity"
        return
    if quantity > self.movie.get_seats():
        self.booking_status = "Failed"
        self.failure_reason = "Seats Not Available"
        return
    self.total_amount = (self.movie.get_ticket_price()* quantity)
    discount_percentage = (self.customer.get_discount_percentage())
    self.discount_amount = (self.total_amount* discount_percentage / 100)

    self.final_amount = (self.total_amount- self.discount_amount)
    payment_successful = (self.customer.pay_from_wallet(self.
    final_amount))

    if not payment_successful:
        self.booking_status = "Failed"
        self.failure_reason = "Low Wallet Balance"
        return
    self.movie.reduce_seats(quantity)
    self.booking_status = "Booking Confirmed"

except ValueError:
    self.booking_status = "Failed"
    self.failure_reason = ( "Seats Input Is Not A Number")

def display_booking_details(self):
    print("\nBooking Details")
    print("Customer Name:", self.customer.name)
    print("Movie Name:", self.movie.movie_name)
    print("Seats Booked:", self.quantity)
    print("Total Amount:", self.total_amount)
    print("Discount Amount:", self.discount_amount)
    print("Final Amount:", self.final_amount)
    print("Booking Status:", self.booking_status)
    print("Failure Reason:", self.failure_reason)

```

6 Creating Booking File Manager

```

[5]: class BookingFileManager:

    def __init__(self, file_name):
        self.file_name = file_name

    def save_booking(self, booking):
        with open(self.file_name, "a") as file:
            file.write("Customer Name: " + booking.customer.name + "\n")

```

```

        file.write("Movie Name: "+ booking.movie.movie_name + "\n")
        file.write("Seats Booked: "+ str(booking.quantity) + "\n")
        file.write("Final Amount: "+ str(booking.final_amount) + "\n")
        file.write("Booking Status: "+ booking.booking_status + "\n")
        file.write("Failure Reason: "+ booking.failure_reason + "\n")
        file.write("-----\n")
    print("Booking saved successfully.")

```

Booking saved successfully.

7 Creating Report Manager

```

[6]: class ReportManager:

    def generate_report(self, bookings):
        total_tickets_booked = 0
        total_revenue = 0

        successful_bookings = {}
        failed_bookings = {}
        remaining_seats = {}

        for booking in bookings:
            movie_name = booking.movie.movie_name
            remaining_seats[movie_name] = (booking.movie.get_seats())

            if booking.booking_status == ("Booking Confirmed"):
                total_tickets_booked += int(booking.quantity)
                total_revenue += (booking.final_amount)

                if movie_name in successful_bookings:
                    successful_bookings[movie_name] += 1
                else:
                    successful_bookings[movie_name] = 1
            else:
                if movie_name in failed_bookings:
                    failed_bookings[movie_name] += 1
                else:
                    failed_bookings[movie_name] = 1

        print("\n===== MOVIE REPORT =====")
        print("Total Tickets Booked:",total_tickets_booked)
        print("Total Booking Revenue:",total_revenue)
        print("Movie Wise Successful Bookings:",successful_bookings)
        print("Movie Wise Failed Bookings:",failed_bookings)
        print("Remaining Seats Report:",remaining_seats)

```

8 Creating Customers and Movies

```
[7]: customer1 = Customer("Deepak",1000)
      customer2 = StudentCustomer("Daya",1000)
      customer3 = PremiumCustomer("Hemant",500)

      movie1 = Movie("Avengers",250,10)
      movie2 = Movie("Interstellar",300,2)

      movie1.display_movie_details()
      movie2.display_movie_details()
```

Movie Details

Movie Name: Avengers

Ticket Price: 250

Available Seats: 10

Movie Details

Movie Name: Interstellar

Ticket Price: 300

Available Seats: 2

9 Creating Booking Objects

```
[8]: booking1 = Booking(customer1, movie1, 2)
      booking2 = Booking(customer2, movie1, 2)
      booking3 = Booking(customer3, movie1, 5)
      booking4 = Booking(customer1, movie2, 10)
      booking5 = Booking(customer1, movie1, "two")
      booking6 = Booking(customer1, movie1, 0)
```

10 Placing Bookings

```
[9]: bookings = [booking1,booking2,booking3,booking4,booking5,booking6]

      for booking in bookings:
          booking.place_booking()
          booking.display_booking_details()
```

Booking Details

Customer Name: Deepak

Movie Name: Avengers

Seats Booked: 2

Total Amount: 500

Discount Amount: 0.0

Final Amount: 500.0
Booking Status: Booking Confirmed
Failure Reason: None

Booking Details
Customer Name: Daya
Movie Name: Avengers
Seats Booked: 2
Total Amount: 500
Discount Amount: 50.0
Final Amount: 450.0
Booking Status: Booking Confirmed
Failure Reason: None

Booking Details
Customer Name: Hemant
Movie Name: Avengers
Seats Booked: 5
Total Amount: 1250
Discount Amount: 250.0
Final Amount: 1000.0
Booking Status: Failed
Failure Reason: Low Wallet Balance

Booking Details
Customer Name: Deepak
Movie Name: Interstellar
Seats Booked: 10
Total Amount: 0
Discount Amount: 0
Final Amount: 0
Booking Status: Failed
Failure Reason: Seats Not Available

Booking Details
Customer Name: Deepak
Movie Name: Avengers
Seats Booked: two
Total Amount: 0
Discount Amount: 0
Final Amount: 0
Booking Status: Failed
Failure Reason: Seats Input Is Not A Number

Booking Details
Customer Name: Deepak
Movie Name: Avengers
Seats Booked: 0


```
Total Amount: 0
Discount Amount: 0
Final Amount: 0
Booking Status: Failed
Failure Reason: Invalid Seat Quantity
```

11 Saving Bookings into File

```
[10]: file_manager = BookingFileManager("bookings.txt")
      for booking in bookings:file_manager.save_booking(booking)
```

12 Final Report

```
[11]: report = ReportManager()
      report.generate_report(bookings)
```

```
===== MOVIE REPORT =====
```

```
Total Tickets Booked: 4
```

```
Total Booking Revenue: 950.0
```

```
Movie Wise Successful Bookings: {'Avengers': 2}
```

```
Movie Wise Failed Bookings: {'Avengers': 3, 'Interstellar': 1}
```

```
Remaining Seats Report: {'Avengers': 6, 'Interstellar': 2}
```

```
[ ]:
```

```
[ ]:
```

```
[ ]:
```